Finals Buddy

Technical Documentation

An AI-powered adaptive finals-prep workspace

Architecture • Data Model • API • AI Agents

Developer / architecture reference

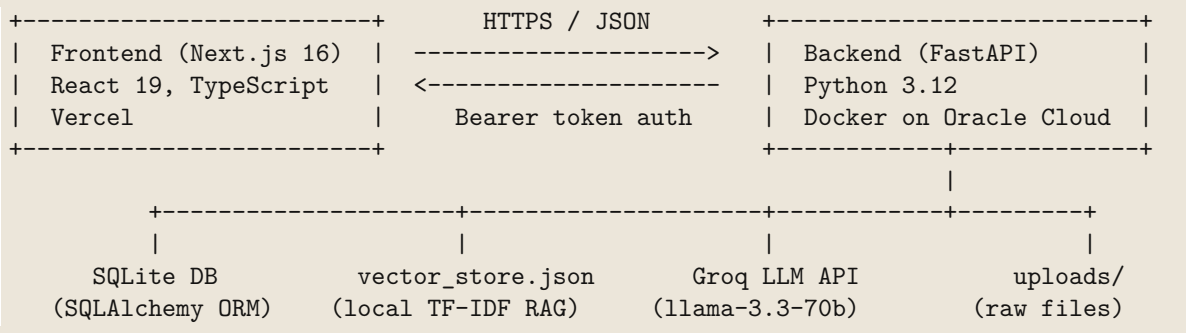
Contents

1 Overview

Finals Buddy is an AI study companion. A user uploads course material (PDF, DOCX, PPTX, TXT); the backend parses it, indexes it for retrieval, and runs a chain of LLM agents to produce a summary, flashcards, quizzes, mock exams, formula sheets, and a knowledge map — all grounded in the user's own material. A RAG tutor lets the user chat with their material, and a recommendation engine ranks what to study next.

For the product pitch, feature list, and screenshots, see the root `README.md`. This document covers how the system is built.

Two-service architecture:



The backend is deliberately self-contained: SQLite for relational data, a file-backed TF-IDF vector store for RAG (no external vector DB), and Groq as the only external AI dependency. Everything that must survive a redeploy lives in one mountable data volume.

2 Tech stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS 4
Rich editor	TipTap 3 (+ math extension, KaTeX)
Icons	lucide-react
Backend	FastAPI, Uvicorn, Python 3.12
ORM / DB	SQLAlchemy 2, SQLite
LLM	Groq `llama-3.3-70b-versatile` (groq SDK + LangChain)
Agent loop	LangChain / langchain-groq
RAG retrieval	Custom local TF-IDF cosine store (`vector_store.py`)
Compression	Headroom (local ONNX text compressor)
Doc parsing	pypdf, docx2txt, python-pptx
Crypto	`cryptography` (Fernet) for BYOK key encryption
Deploy	Docker → Oracle Cloud (Ampere ARM64); frontend on Vercel

Note on Next.js. `frontend/AGENTS.md` warns this is a newer Next.js with breaking changes vs. older conventions — consult `node_modules/next/dist/docs/` before writing frontend code.

3 Repository layout

```
finals-buddy/
```

```
|-- backend/
|   |-- app/
|   |   |-- main.py          # FastAPI app + all HTTP endpoints (~1700 lines)
|   |   |-- models.py        # SQLAlchemy ORM models (the data model)
|   |   |-- schemas.py       # Pydantic request/response schemas
|   |   |-- database.py      # engine, SessionLocal, get_db dependency
|   |   |-- config.py        # env config + runtime Groq-key overrides
|   |   |-- auth.py          # PBKDF2 hashing + HMAC bearer tokens
|   |   |-- key_context.py   # BYOK + free-trial gating (contextvar routing)
|   |   |-- admin.py         # /api/admin/* router
|   |   |-- errors.py        # AIServiceError
|   |   `-- services/
|   |       |-- agents.py         # LLM agents (summarize, quiz, exam, ...)
|   |       |-- langchain_chat.py # RAG tutor (Groq tool-calling loop)
|   |       `-- vector_store.py   # local TF-IDF vector store
|   |-- Dockerfile
|   `-- requirements.txt
|-- frontend/
|   `-- src/
|       |-- app/
|       |   |-- page.tsx              # home / subject list
|       |   |-- login/page.tsx
|       |   |-- admin/page.tsx
|       |   `-- subject/[id]/page.tsx # main study workspace (~3800 lines)
|       |-- components/               # AccountSettings, NotionEditor, Toast
|       `-- lib/api.ts                # typed API client + session handling
|-- docs/                             # screenshots + this document
`-- README.md
```

4 Data model

All tables are defined in `backend/app/models.py`. `User` owns `Subject`s; almost everything else hangs off `Subject` with `ON DELETE CASCADE`.

```
User --< Subject --< Material --< Quiz
                 |--< Flashcard          (Leitner box 1-5, spaced repetition)
                 |--< Task --1 Recommendation
                 |--< StudySession
                 |--< Quiz
                 |--< ChatMessage        (tutor history)
                 |--< MockExam --< MockExamQuestion
                 |--< Formula
                 |--< Note               (TipTap rich content)
                 `--< ResourceConnection (edges of the knowledge map)
```

Key fields worth knowing:

- **User** — `groq_api_key_encrypted` (Fernet-encrypted BYOK key, NULL = still on trial), `trial_requests_used` (counts free AI actions).
- **Subject** — `exam_date`, `priority_level` (1–5), `difficulty` (1–5), `confidence_score` (0–100). These feed the recommendation formula.
- **Material** — `summary`, `key_concepts` (JSON), `learning_complexity`, `importance_level`, `deep_research_summary`.

- **Flashcard** — `box` (Leitner 1–5) + `next_review_date` drive spaced repetition.
- **ResourceConnection** — `source_material_id` → `target_material_id` with a `connection_type` (Prerequisite / Extension / Foundational): the AI-generated knowledge-map edges.

4.1 Schema migrations

There is no Alembic. On startup `main.py` runs `Base.metadata.create_all()` to create missing tables, followed by a series of idempotent `ALTER TABLE ... ADD COLUMN` statements wrapped in try/except (e.g. adding `deep_research_summary`, `groq_api_key_encrypted`, `trial_requests_used`). This lightweight approach keeps the SQLite file forward-compatible across redeploys without a migration tool.

5 Authentication

`backend/app/auth.py`, intentionally dependency-free (hashlib / hmac / secrets only):

- **Passwords:** PBKDF2-HMAC-SHA256, 260,000 iterations.
- **Tokens:** `base64url(payload).hex(hmac_sha256(secret, payload))`, 30-day TTL. Stateless — no server session store.
- **Secret:** `SECRET_KEY` from env, else generated once and persisted to `.auth_secret` next to the DB so sessions survive restarts.
- **Frontend:** token + user cached in `localStorage` (`fb_token` / `fb_user`). `authFetch` in `lib/api.ts` attaches the bearer token and, on a 401, clears the session and redirects to `/login`.
- **Admin:** `ADMIN_EMAILS` env allowlist gates `/api/admin/*`. Single-owner project — a plain email allowlist, no roles system.

6 AI agent layer

`backend/app/services/agents.py`. Every agent calls `run_llm()`, which targets Groq `llama-3.3-70b-versatile` at `temperature=0.3`, optionally in JSON mode, and raises `AIServiceError` on any failure rather than fabricating content. Before the prompt is sent, Headroom compresses the user-supplied context locally to cut Groq token spend.

Agent	Job
SummarizationAgent	Exhaustive cited summary + key concepts + complexity / importance ratings (JSON). Emits Mermaid/ASCII diagrams and `[Slide N]` citations.
DeepResearchAgent	Under-the-hood mechanics, exam pitfalls, real-world case studies — Markdown enrichment supplement.
QuizGenerationAgent	Flashcards + multiple-choice quizzes with mandatory source citations and per-question explanations; `generate_more_items` adds more while avoiding duplicates.
MockExamAgent	Generates 3 open-ended exam questions; grades typed answers 0–100 on accuracy / completeness / terminology with constructive feedback.
FormulaExtractorAgent	Extracts formulas as LaTeX + variables + derivation steps (JSON).
CurriculumMapperAgent	Computes typed conceptual edges between materials → the knowledge map.
PlanningRecommendationAgent	*Not an LLM call* — deterministic Python scoring of tasks (see below).

6.1 Recommendation score

`PlanningRecommendationAgent.calculate_recommendations` ranks pending tasks:

```
score = (exam_urgency  * 0.4) + (topic_importance * 0.3)
      + (low_confidence * 0.2) + (incompletion    * 0.1)
```

Urgency is derived from days remaining to `exam_date` ($\leq$1 day $\rightarrow$ 10, decaying with distance); low-confidence from `100 - confidence_score`. The result is scaled to 0–100 for display. The lowest-confidence topic with the nearest exam floats to the top.

7 Ingestion pipeline

`POST /api/materials/upload` (`main.py`). Upload is a 7-step pipeline; steps 1–2 run synchronously in the request, steps 3–7 run in a FastAPI `BackgroundTask` so the HTTP response returns immediately with a `job_id`.

```
[1] Save file to uploads/                          (sync, in request)
[2] Extract text (pypdf / docx2txt / python-pptx)  (sync, in request)
    -- response returns with job_id --
[3] Chunk (1000 chars, 200 overlap) -> index into vector_store.json
[4] SummarizationAgent -> summary, key_concepts, ratings
[5] DeepResearchAgent  -> deep_research_summary   (dashboard unlocks here)
[6] QuizGenerationAgent -> flashcards + quizzes
[7] Create a "review" study Task
```

Progress streams to the client over **Server-Sent Events** at `GET /api/materials/upload-progress/{job_id}`. The free-trial action is consumed *only after* the background pipeline succeeds, so a failed digestion never burns a user's allowance.

Because the background thread runs after the response is sent, it cannot see the request's

contextvar; the resolved Groq key is passed explicitly and re-bound with `set_request_key()` inside the thread (see §9).

8 RAG tutor

`backend/app/services/langchain_chat.py`, entered from `POST /api/tutor/chat`.

- **Retrieval:** `vector_store.py` is a from-scratch **TF-IDF cosine** store persisted as `vector_store.json` — no external vector DB, no embedding API. It computes IDF over the filtered document set at query time and returns the top-k chunks. Documents are filtered by `subject_id` so a query only ever sees the current subject's material.
- **Agent loop:** a Groq tool-calling loop (max 5 iterations) with three tools — `search_course_materials`, `get_subject_info`, `get_study_progress`. Search is hard-capped at 2 calls per message.
- **Token compression:** retrieved chunks are run through Headroom (`target_ratio=0.2`) before going back to Groq — RAG chunks are search results, not conversation, so aggressive compression is safe and cuts cost.
- **Personas (`mode`):** `standard`, `simplified` ("explain like I'm 5"), and `analogies` inject different system-prompt suffixes.
- **Resilience:** on a malformed tool call the loop retries at escalating temperature (greedy decoding would just reproduce the same bad call); if tool-calling can't be recovered it falls back to a direct no-tools answer. In trial mode only, a 429 rate-limit spills over to a second server key (`GROQ_API_KEY_2`) — a personal key never spills onto the owner's account.

9 BYOK + free-trial model

`backend/app/key_context.py`. The app used to share one server Groq key for every visitor; this module makes AI usage freemium so hosting cost stays flat.

- New accounts get `FREE_TRIAL_LIMIT = 5` AI actions on the **server** key.
- After that, the user saves a personal Groq key (via `PUT /api/account/groq-key`), which is validated with a live `models.list()` call and stored **Fernet-encrypted** at rest. The Fernet key is derived (SHA-256) from the app's existing `SECRET_KEY`, so there is no new secret to manage. Keys are never returned in full — only a masked hint (`gsk_...4f9a`).
- **Key routing:** the resolved per-request key is stashed in a `contextvars.ContextVar`; `agents.run_llm` and the tutor read it via `get_current_key()`. This threads a per-user key through module-level Groq client singletons without rewriting every agent signature. `contextvars` are per-thread/task, so it is safe under FastAPI's sync threadpool and async workers.
- **Gate:** `ai_action()` (context manager) resolves the key, enforces the cap (HTTP 402 when exhausted, 503 when no key exists), and increments the trial counter *only* on success. `count=False` covers auto-fired paths like recommendations that should use a key but not consume the allowance.

9.1 Runtime key overrides

`config.py` supports a `.runtime_overrides.env` file: the admin dashboard's "rotate Groq key" feature writes here (in the persisted data volume) instead of editing the container's real `.env`, so key changes survive `docker rm` + redeploy without rebuilding the image.

10 HTTP API reference

All routes are under `/api`. All except signup/login require a bearer token. Ownership is enforced per subject (`own_subject` / `assert_owner`).

10.1 Auth & account

Method	Path	Purpose
POST	`/auth/signup`	Register, returns token
POST	`/auth/login`	Log in, returns token
GET	`/auth/me`	Current user
GET	`/account`	Account + trial/key status
PUT	`/account/groq-key`	Save/validate personal Groq key
DELETE	`/account/groq-key`	Remove personal key (back to trial)

10.2 Subjects

Method	Path	Purpose
POST	`/subjects`	Create subject
GET	`/subjects`	List subjects
GET	`/subjects/{id}`	Subject dashboard payload
PATCH	`/subjects/{id}`	Update (exam date, priority, confidence...)
DELETE	`/subjects/{id}`	Delete (cascades)

10.3 Materials & knowledge map

Method	Path	Purpose
POST	`/materials/upload`	Upload + start ingestion, returns `job_id`
GET	`/materials/upload-progress/{job_id}`	SSE progress stream
GET	`/subjects/{id}/materials`	List materials
PATCH/DELETE	`/materials/{id}`	Edit / delete material
POST	`/subjects/{id}/generate-map`	Build knowledge-map edges
GET	`/subjects/{id}/map`	Fetch nodes + edges

10.4 Study aids

Method	Path	Purpose
GET/POST	`/subjects/{id}/flashcards`	List / add flashcards
PUT/DELETE	`/flashcards/{id}`	Edit / delete
POST	`/flashcards/{id}/review`	Record review, advance Leitner box
GET/POST	`/subjects/{id}/quizzes`	List / add quizzes
PUT/DELETE	`/quizzes/{id}`	Edit / delete
POST	`/quizzes/{id}/answer`	Submit answer, get grading
POST	`/subjects/{id}/generate-more`	Generate more flashcards/quizzes
*	`/subjects/{id}/mock-exams`, `/mock-exams/{id}`	Mock exams (CRUD)
POST	`/mock-exams/{id}/submit`	Submit + auto-grade
GET/POST	`/subjects/{id}/formulas`, `/formulas/generate`	Formula sheets
*	`/formulas/{id}`	Edit / delete / annotate

10.5 Tasks, sessions, notes, tutor, dashboard

Method	Path	Purpose
*	`/tasks`, `/subjects/{id}/tasks`, `/tasks/{id}`	Planner tasks
*	`/study-sessions`, `/subjects/{id}/sessions`	Focus sessions
*	`/subjects/{id}/notes`, `/notes/{id}`	Rich notes
POST	`/tutor/chat`	RAG tutor (query + persona mode)
*	`/subjects/{id}/chats`, `/chats/{id}`	Tutor history
GET	`/dashboard/recommendations`	Ranked next actions
GET	`/dashboard/summary`	Aggregate stats

10.6 Admin (`ADMIN_EMAILS` only)

`GET /admin/overview`, `GET /admin/logs`, `GET /admin/health`, `GET|PUT /admin/config` — usage stats, app logs (from the stdout tee), health checks, and live Groq-key rotation.

11 Frontend

Next.js App Router. Three main routes:

- `/` **(page.tsx)** — auth gate + subject list/creation.
- `/subject/[id]` **(page.tsx, ~3800 lines)** — the study workspace: dashboard, materials, flashcards, quizzes, mock exams, formula sheets, knowledge map, notes, tutor chat, focus mode. This is where most of the product lives.
- `/admin` — admin dashboard (usage, logs, health, key rotation).

`lib/api.ts` is the single typed API client: session helpers, `authFetch` (bearer + 401 redirect), and `ApiError` (carries HTTP status so callers can branch, e.g. 403 → "not authorized").

`API_BASE` comes from `NEXT_PUBLIC_API_URL` (defaults to `http://127.0.0.1:8000/api`). Notes use the TipTap-based `NotionEditor` with math/KaTeX support.

12 Configuration

Backend env (`backend/.env`, see `.env.example`):

Var	Purpose
`GROQ_API_KEY`	Server key for the free trial
`GROQ_API_KEY_2`	Rate-limit fallback (trial mode only)
`DATABASE_URL`	Defaults to `sqlite:///./finals_buddy.db`
`SECRET_KEY`	Token signing + BYOK encryption seed (auto-generated if unset)
`CORS_ORIGINS`	Comma-separated allowed frontend origins
`ADMIN_EMAILS`	Comma-separated admin allowlist
`RUNTIME_OVERRIDES_PATH`	Path to the runtime key-override file
`PORT / HOST`	Server bind (default `0.0.0.0:8000`)

Frontend env: `NEXT_PUBLIC_API_URL` → backend `/api` base.

13 Running locally

Backend:

```
cd backend
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\
    activate
pip install -r requirements.txt
cp .env.example .env        # add GROQ_API_KEY
python -m uvicorn app.main:app --reload --port 8000
```

API docs (Swagger) at `http://127.0.0.1:8000/docs`.

Frontend:

```
cd frontend
npm install
# .env.local: NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
npm run dev        # http://localhost:3000
```

14 Deployment

- **Backend:** Docker image (`backend/Dockerfile`), `python:3.12-slim`, runs on x86_64 and ARM64 (Oracle Cloud Ampere A1). The app *runs from* `/srv/finals-buddy/data` (its `WORKDIR`) while code lives at `/srv/finals-buddy`, so every relative artifact — `finals_buddy.db`, `uploads/`, `vector_store.json`, `.auth_secret`, `.runtime_overrides.env`, `logs/app.log` — lands in one mountable volume:

  ```
  docker run -v finals_data:/srv/finals-buddy/data --env-file .env \
             -p 8000:8000 finals-buddy
  ```

Uvicorn runs with `-proxy-headers -forwarded-allow-ips=*` behind a reverse proxy.

- **Frontend:** Vercel; `NEXT_PUBLIC_API_URL` points at the deployed backend.
- **Logs:** stdout/stderr are teed to `logs/app.log` (size-capped) so the admin dashboard can show `docker logs`-equivalent output without mounting the Docker socket into the container.

For the full deployment runbook see `DEPLOY.md` (not tracked in the public repo).